

Reviewer Pack — Minimal Rank of Noncommutative Hopf Algebras over Boolean Fu...

Entry cd84f3cc2ccf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 11:17:02 UTC

Minimal Rank of Noncommutative Hopf Algebras over Boolean Functions vs Sum-of-Squares Approximation Ratio for Max-CUT

Entry ID: cd84f3cc2ccf **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 11:17:02 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra (Hopf Algebras) **Field B** (complexity object): Complexity Theory: Sum-of-Squares Complexity (for Max-CUT)

Statement:

[‘For a given instance of the Max-CUT problem with n vertices, there exists a noncommutative Hopf algebra H associated with the Boolean function defining the instance such that the minimal rank of H is upper bounded by a constant factor of the optimal sum-of-squares approximation ratio.’, ‘The minimal rank of any Hopf algebra associated with an instance of Max-CUT is at most 2.5 times the optimal sum-of-squares approximation ratio for Max-CUT.’]

Rationale (proposer’s reasoning):

[‘Hopf algebras provide a framework to encode complex Boolean functions and their interactions. The minimal rank of a Hopf algebra could potentially capture essential properties of the function that are related to its complexity, such as the approximation ratio in sum-of-squares complexity for Max-CUT.’, ‘This conjecture bridges noncommutative algebra with complexity theory by leveraging the structure of Hopf algebras to provide insights into the computational hardness of Max-CUT.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 034f7f83d062e7ae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of the associated noncommutative Hopf algebra is upper bounded by a constant factor of the optimal sum-of-squares approximation ratio for Max-CUT, specifically if the ratio of minimal rank to optimal approximation ratio ≤ 2.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative Hopf algebra" AND "sum-of-squares approximation ratio" AND Max-CUT - "minimal rank" IN Hopf algebras AND application "sum-of-squares complexity" FOR Max-CUT - "Boolean function" IN noncommutative algebra AND connection "with" complexity theory"

Top relevant hits considered: - [http://arxiv.org/abs/1707.02822v3] Discriminants of Taft algebra smash products and applications - [http://arxiv.org/abs/math/0702755v1] Restricted simple Lie algebras and their infinitesimal deformations - [http://arxiv.org/abs/2401.03308v3] On von Neumann regularity of ample groupoid algebras - [http://arxiv.org/abs/0705.1265v2] A noncommutative Bohnenblust-Spitzer identity for Rota-Baxter algebras solves Bogoliubov's recursion - [http://arxiv.org/abs/2411.11095v3] Invariant theory and coefficient algebras of Lie algebras - [http://arxiv.org/abs/math/0610043v5] Lecture Notes on Noncommutative Algebraic Geometry and Noncommutative Tori

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def boolean_function(instance):
    n = len(instance)
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    return [instance[edges.index((i, j))] for i, j in edges]

def hopf_algebra_rank(boolean_func):
    # Simplified encoding of a Hopf algebra rank based on the Boolean function
    # This is a placeholder and should be replaced with actual computation
    return len(boolean_func)

def max_cut_approximation_ratio(instance):
    n = len(instance)
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    cut_value = sum(instance[edges.index((i, j))] for i, j in edges if random.choice([0, 1]) == 0)
    return Fraction(cut_value, len(edges))
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random instance of Max-CUT with n vertices
    n = random.randint(5, 40)
    instance = [random.choice([0, 1]) for _ in range(n * (n - 1) // 2)]

    boolean_func = boolean_function(instance)
    hopf_rank = hopf_algebra_rank(boolean_func)
    approx_ratio = max_cut_approximation_ratio(instance)

    if approx_ratio == 0:
        return {
            "metric_name": "ratio",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "approx_ratio_is_zero"
        }

    ratio = hopf_rank / approx_ratio

    return {
        "metric_name": "ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": ratio <= 2.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results if r["instances_tested"] > 0) / len(results)
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results if r["instances_tested"] > 0) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='approx_ratio_is_zero' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cb5b0c9.py", line 70, in <module>
    result = run_trial(seed)
            ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cb5b0c9.py", line 41, in run_trial
    boolean_func = boolean_function(instance)
                  ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cb5b0c9.py", line 21, in boolean_func
    return [instance[edges.index((i, j))] for i, j in edges]
            ~~~~~^~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result. | next: Re-run the test with proper error handling and input validation to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11788
2	preregistration	ollama_remote	glm4:latest	0	0	5367
3	novelty	ollama_remote	glm4:latest	0	0	4909
4	novelty	ollama_remote	glm4:latest	0	0	5990
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15129
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7695
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8201
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8798
9	judge	ollama_remote	glm4:latest	0	0	8038

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 75914 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cd84f3cc2ccf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cd84f3cc2ccf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cd84f3cc2ccf.tar.gz` (if generated)